

benchmark_baseline_ratchet_test.sh

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z

Standing §11.4.135 regression guard for the F6 benchmark-baseline-ratchet fix. It proves the dynamic benchmark suite's §11.4.169(13) performance ratchet genuinely compares each run's measured p95 latency against a **recorded, committed baseline** — a regressed measurement FAILs, an in-tolerance one PASSes, and a first run with no baseline SEEDs-and-SKIPs instead of emitting a silent budget-only PASS.

Overview

Field	Value
Path	tests/regression/benchmark_baseline_ratchet_test.sh
Under test	tests/dynamic/suites/benchmark_suite.sh → bench_regression_verdict()
Baseline file	tests/dynamic/baselines/benchmark_p95.baseline (committed, tracked)
Kind	Fixture-driven pure-function guard — no network, no containers, no live benchmarking
Anti-bluff anchors	§11.4.169(13), §11.4.1, §11.4.115, §11.4.135, §1.1
Exit codes	0 = PASS, 1 = FAIL

Prerequisites

- POSIX sh, plus bash, awk, mktemp on PATH.
- The tracked suite tests/dynamic/suites/benchmark_suite.sh must define the column-0 pure function bench_regression_verdict() (the guard awk-extracts and drives the REAL function — it does not re-implement it).
- No live dynamic stack, no baseline file, and no credentials are required — the guard is entirely fixture-driven.

Usage examples

```
# GREEN standing guard (default): asserts the real ratchet
# classifies correctly.
tests/regression/benchmark_baseline_ratchet_test.sh

# RED reproduction ($11.4.115): asserts the pre-fix budget-only
# logic wrongly
# PASSES a regressed-but-within-budget measurement (the bluff
# reproduced).
RED_MODE=1 tests/regression/benchmark_baseline_ratchet_test.sh
```

Run under the mandated host caps:

```
GOMAXPROCS=2 nice -n 19 ionice -c 3 tests/regression/
benchmark_baseline_ratchet_test.sh
```

What it asserts

The guard extracts the REAL `bench_regression_verdict()` from the tracked suite and drives it with fixed tuples (`p95_ms`, `budget_ms`, `baseline_ms`, `regress_pct`), baseline 800ms, budget 2000ms, tolerance 25%:

Tuple	Meaning	Expected verdict
850 2000 800 25	measured within budget, +6% vs baseline	PASS
1200 2000 800 25	measured within budget, +50% vs baseline	FAIL:regression
850 2000 0 25	no baseline recorded yet	SEED
2500 2000 800 25	over the absolute budget	FAIL:budget

The decisive anti-bluff assertion is `REGRESS=FAIL:regression`: any change that lets a regressed measurement PASS (the pre-fix behaviour) flips it to PASS and FAILs the GREEN guard with a real assertion mismatch — not a parse error.

`RED_MODE=1` runs a faithful pre-fix replica (`p95 <= budget` only, no baseline comparison — modelling the never-present gitignored-throwaway baseline) against the 1200 measurement and asserts it PASSES, reproducing the ratchet bluff.

Edge cases

- **Non-numeric / zero p95** → FAIL:budget (an unmeasured run is never a PASS).
- **Empty or 0 baseline** → SEED (caller seeds the committed file + SKIPS).
- **Improvement (p95 < baseline)** → negative growth → PASS.
- **RED cannot reproduce** → the guard reports a §11.4.7 finding rather than a false PASS.

Internal behaviour

1. Resolve `REPO_ROOT` from the script location; create the gitignored evidence dir `qa-results/regression/benchmark_baseline_ratchet/`.
2. Build a probe script in a `mktemp` file (cleaned via `trap ... EXIT`):
 - `RED_MODE=1` → a pre-fix budget-only replica + one driver line.
 - `RED_MODE=0` → `awk-extract bench_regression_verdict()` from the tracked suite + four driver lines.
3. Run the probe under `bash`, capture `stdout+stderr` and `rc`.
4. Pattern-match the captured output; write a structured evidence file; print a single `[PASS|FAIL]` verdict line + the evidence path; exit `0/1`.

§1.1 paired-mutation proof

Mutate `bench_regression_verdict()` in the suite so a regressed number wrongly PASSES (e.g. force the `FAIL:regression` branch to never fire). The GREEN guard then FAILs on `REGRESS=PASS` (a real assertion mismatch). Restore the suite byte-identical (md5 match) and the GREEN guard PASSES again. This proves the guard is not a bluff gate.

Related scripts

- `tests/dynamic/suites/benchmark_suite.sh` — the suite whose ratchet this guards.
- `tests/dynamic/baselines/README.txt` — the committed-baseline seed/arm workflow.
- `tests/regression/external_egress_verdict_test.sh` — sibling pure-function extraction guard (the house pattern this follows).
- `tests/lib/evidence.sh` — canonical `ab_pass_with_evidence / ab_skip_with_reason`.

Last verified

2026-07-01 — GREEN PASS, RED_MODE=1 PASS (reproduction), and the §1.1 mutation→FAIL→restore(md5-match) cycle captured by the F6 fix author.